

PRACTICAL_04

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions. Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

OUTPUT 1:-

```
v_santosh@DESKTOP-E34M7RI:~$ nano Practical_04_1.c
v_santosh@DESKTOP-E34M7RI:~$ gcc Practical_04_1.c -o Practical_04_1
v_santosh@DESKTOP-E34M7RI:~$ ./Practical_04_1
Parent Process: PID = 763
Child 1: PID = 764, Parent PID = 763
Child 2: PID = 765, Parent PID = 763

Using wait():
Child 3: PID = 766, Parent PID = 763
Child 1 finished
wait(): Child PID 764 completed with status 10

Using waitpid():
Child 2 finished
waitpid(): Child PID 765 completed with status 11

All child processes synchronized.
v_santosh@DESKTOP-E34M7RI:~$ Child 3 finished
```

CODE FOR OUTPUT 1:-

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child[3];
    int status;

    printf("Parent Process: PID = %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++)
    {
        child[i] = fork();

        if (child[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (child[i] == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(2 + i);

            printf("Child %d finished\n", i + 1);
            exit(10 + i);
        }
    }

    // wait() - waits for any child
    printf("\nUsing wait():\n");

    pid_t pid = wait(&status);

    if (WIFEXITED(status))
        printf("wait(): Child PID %d completed with status %d\n",
            pid, WEXITSTATUS(status));

    // waitpid() - waits for a specific child
    printf("\nUsing waitpid():\n");

    for (int i = 0; i < 2; i++)
    {
        pid = waitpid(child[i], &status, 0);

        if (pid > 0 && WIFEXITED(status))
            printf("waitpid(): Child PID %d completed with status %d\n",
                pid, WEXITSTATUS(status));
    }

    printf("\nAll child processes synchronized.\n");

    return 0;
}
```

OUTPUTS 2:-

```
v_santosh@DESKTOP-E34M7RI:~$ gcc zombie_fixed.c -o zombie_fixed
v_santosh@DESKTOP-E34M7RI:~$ ./zombie_fixed
Parent process: PID = 984
Child PID = 985
Child process: PID = 985
Child exiting...
Child completed successfully.
Exit status = 0
No zombie process exists.
```

```
v_santosh@DESKTOP-E34M7RI:~$ nano zombie.c
v_santosh@DESKTOP-E34M7RI:~$ gcc zombie.c -o zombie
v_santosh@DESKTOP-E34M7RI:~$ ./zombie &
[2] 938
Parent process: PID = 938
Child PID = 940
Parent sleeping. Child becomes a zombie.
Child process: PID = 940
Child exiting...
v_santosh@DESKTOP-E34M7RI:~$ ps -el | grep zombie
0 S 1000 938 335 0 80 0 - 692 hrtime pts/0 00:00:00 zombi
1 Z 1000 940 938 0 80 0 - 0 - pts/0 00:00:00 zombi
v_santosh@DESKTOP-E34M7RI:~$ Parent exiting...
nano zombie_fixed.c
[2]- Done ./zombie
```

CODES FOR OUTPUT 2:-

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping. Child becomes a zombie.\n");

        sleep(20);

        printf("Parent exiting...\n");
    }

    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child completed successfully.\n");
            printf("Exit status = %d\n", WEXITSTATUS(status));
        }

        printf("No zombie process exists.\n");
    }

    return 0;
}
```